

Modelos e Técnicas Computacionais para Análise de Requisitos de Segurança (Safety e Security)

Prof. Rodrigo Martins Pagliares — Unifal

PARTE 1 — RESUMO COMPLETO DA MATÉRIA

1. Requisitos de Software — SWEBOK

1.1 Fundamentos

Requisito de software é uma condição ou capacidade que um usuário precisa para resolver um problema ou atingir um objetivo, e que deve estar presente no sistema. A área é dividida em: **Elicitação** → **Análise** → **Especificação** → **Validação** → **Gestão**.

Categorias de Requisitos:

- **Funcionais:** descrevem o que o sistema deve fazer (algoritmos, regras de negócio) — independentes de tecnologia.
- **Não funcionais:** qualidades do sistema. Subdividem-se em:
 - *Restrições tecnológicas* (plataforma, linguagem, BD).
 - *Qualidade de serviço* — usabilidade, confiabilidade, disponibilidade, escalabilidade, manutenibilidade.
- **Requisitos de projeto:** restrições de prazo, custo e escopo (presentes no Project Charter).
- **Safety e Security** são requisitos **não funcionais** — propriedades **emergentes** que surgem da interação dos componentes.

Filtro da tecnologia perfeita: tudo o que você ainda precisaria especificar mesmo numa máquina perfeita = requisito funcional. A máquina perfeita em si = requisito não funcional.

1.2 Elicitação

Investigação inicial para identificar requisitos. Fontes: stakeholders, normas (ISO/IEC), legislação, versões anteriores do software, benchmarking competitivo.

Técnicas: entrevistas, workshops (JAD/JRP), brainstorming, questionários, prototipação (low/high fidelity), User Story Mapping, observação, estudo de casos.

1.3 Análise

- Cada requisito deve ser: **não ambíguo, testável, necessário, viável, correto e representativo** das necessidades dos stakeholders.
- A coleção deve ser: **completa, consistente internamente e externamente, e viável**.
- **Five Whys:** técnica para descobrir se um requisito é realmente necessário.
- **Qualidade de serviço:** existe um ponto de melhor custo-benefício entre performance e valor para o cliente.
- **Resolução de conflitos:** negociação entre stakeholders; técnica Product Family Development.

1.4 Especificação

Formas de documentar requisitos:

- **Linguagem natural não estruturada:** "O sistema deverá..." — comum na indústria aeronáutica.
- **Linguagem natural estruturada:** casos de uso (UML), histórias de usuário ("Enquanto... Eu quero... Para...").
- **CrITÉrios de aceitação (ATDD/BDD):** sintaxe Gherkin — Given / When / Then → automatizáveis com Cucumber.
- **Modelos gráficos:** diagrama de caso de uso (UML), SysML, DFD, ER, diagramas de estado.
- **Especificação formal:** linguagem Z — precisa, provável matematicamente, menos ambígua.
- **Atributos típicos:** ID único, título, prioridade, risco, fonte, justificativa (rationale), histórico, rastreabilidade.

1.5 Validação

Verificação: "construir certo a coisa" (está de acordo com a especificação?). **Validação:** "construir a coisa certa" (atende as reais necessidades dos stakeholders?). Métodos: revisões, simulação/execução de cenários, prototipação.

1.6 Gestão de Requisitos

- **Requirements Scrubbing:** selecionar o menor subconjunto de requisitos que atende as necessidades (eliminar o desnecessário).
- **Controle de alterações:** em cascata = processo formal rígido; em ágil = priorização no Product Backlog ("Embrace the Change").
- **Scope matching:** garantir que o escopo não exceda custo, prazo e equipe.
- **Rastreabilidade:** forward (req → design/código) e backward (código → req). Fundamental para análise de impacto.
- **Volatilidade:** requisitos propensos a mudanças — use design para interfaces (princípio de inversão de dependência).

1.7 Ferramentas

Ferramentas de **gestão** (IBM DOORS, Jira), **modelagem** (ferramentas UML/SysML), **geração de casos de teste** (Cucumber + Gherkin, diagramas de estado). Essenciais em sistemas complexos onde rastreabilidade e controle de alteração manual são inviáveis.

2. Engenharia de Sistemas e Engenharia de Requisitos

2.1 Conceitos Fundamentais

- **Sistema:** conjunto de componentes que interagem para atingir um objetivo comum. O todo é maior que a soma das partes.
- **Engenharia de Sistemas (SE):** abordagem integrativa e transdisciplinar para habilitar a realização de sistemas (ISO/IEC/IEEE 15288).
- **Modelo V (V Model):** modelo de ciclo de vida mais usado em SE. As atividades não são estritamente sequenciais — ocorrem em paralelo.
- **INCOSE:** conselho internacional que dita normas de SE. Certificações: ASEP → CSEP → ESEP.
- **Propriedades emergentes:** surgem da interação dos componentes — não pertencem a nenhum componente isolado. Ex.: usabilidade, Safety, Security.

- **Necessidades (needs):** declaradas no vocabulário do negócio. **Requisitos:** derivados das necessidades, mais formais e validáveis.
- **Capability Gap:** diferença entre a capacidade atual e a capacidade desejada de um sistema — ponto de partida do Design Conceitual.

2.2 Hierarquia de Documentos (Requisitos)

Documento	Nível	Conteúdo
BRS — Business Requirements Specification	Negócio	Requisitos de negócio
StRS — Stakeholder Requirement Specification	Stakeholders	Necessidades e requisitos dos stakeholders
SyRS — System Requirements Specification	Sistema	Requisitos do sistema (prontos para desenvolvimento)
Sub-SyRS	Subsistema	Requisitos de subsistemas

2.3 Verificação e Validação (V&V;)

Verificar = processo correto; Validar = coisa certa. Testes ocorrem em **todo o ciclo de vida**, não apenas no final.

3. Definições: Safety, Security e Acidente

3.1 Definições Centrais

- **Acidente / Perda (Loss):** evento não planejado que resulta numa perda inaceitável para um stakeholder. Inclui: ferimentos, mortes, danos ambientais, perda de missão, perda financeira/reputação.
- **Safety:** ausência de perdas. Relaciona-se com os **objetivos** e **restrições** do sistema.
— Foco: prevenção de perdas físicas, operacionais ou ambientais causadas por eventos *não intencionais*.
- **Security:** proteção contra ameaças *intencionais* que comprometam **confidencialidade, integridade e disponibilidade (CIA)** do sistema.
- **Hazard (condição de perigo):** estado ou conjunto de condições do sistema que, em combinação com o pior caso do ambiente, leva a uma perda.
— Hazards ≠ Perdas: hazards estão sob controle do projetista; perdas podem envolver o ambiente.
— Hazards ≠ Causas: causas (ex.: falha de sensor) levam a hazards, não são hazards.
- **Restrição de sistema:** comportamento ou condição que o sistema deve satisfazer para prevenir hazards.

3.2 Tabela Comparativa Safety × Security

Aspecto	Safety	Security
Foco	Perdas físicas/operacionais/ambientais	Ameaças intencionais à CIA
Agente causador	Falhas acidentais, erros de design	Agentes maliciosos (spoofing, alteração)
Exemplo de sistema	Controle de tráfego aéreo, usina nuclear	Sistemas financeiros, hospitalares
Objetivo	Ausência de perdas	Confidencialidade, Integridade, Disponibilidade

4. Modelos de Causalidade: STAMP e a Técnica STPA

4.1 Conceitos de Controle (STAMP)

- **STAMP (Systems-Theoretic Accident Model and Processes):** modelo de causalidade que trata safety como problema de controle, não de confiabilidade.
- **Control loop:** composto por Controlador + Processo Controlado, interagindo via **Control Actions** (setas para baixo) e **Feedback** (setas para cima).
- **Componentes do controlador:** Algoritmo de Controle + Modelo de Processo.
→ Para humanos: *modelo mental* (≈ modelo de processo) e *procedimentos operacionais* (≈ algoritmo).
- **Estrutura de controle hierárquica:** conjunto de control loops. Controladores no topo têm maior autoridade.
- **Tipos de controle em nível de design:** fail-safe, interlocks, manutenção, procedimentos, controles culturais.
- **Técnicas de design para Safety:** design intrinsecamente seguro, design passivo, design ativo.
- **Modelo de processo inadequado:** controlador age com informações incompletas ou incorretas sobre o estado real do sistema → ações inseguras.
- **Controladores humanos na STPA:** são tratados como controladores ativos com responsabilidades, modelos mentais e interações que influenciam o comportamento do sistema.

4.2 STPA — 4 Passos

Passo	Nome	O que faz / Saídas
1	Definir o objetivo da análise	• Identificar Perdas (Losses) • Identificar Hazards em nível de sistema • Definir Restrições em nível de sistema • (Opcional) Refinar hazards em sub-hazards
2	Modelar a Estrutura de Controle	• Criar diagrama hierárquico de control loops • Identificar controladores, control actions e feedback • Derivar responsabilidades a partir das restrições
3	Identificar UCAs (Unsafe Control Actions)	• 4 tipos: não fornecida, fornecida quando não devia, momento inadequado, duração inadequada • Rastreáveis aos hazards e perdas do Passo 1 • Geram requisitos funcionais e restrições
4	Identificar Cenários de Perdas	• Causas pelas quais UCAs ocorrem • Feedback incorreto, requisitos ambíguos, erros de design • Gera requisitos adicionais, recomendações de design e casos de teste

4.3 Passo 1 em Detalhe

- **Perdas (Losses):** ferimentos/mortes, dano a propriedade, poluição ambiental, perda de missão, perda de reputação, vazamento de dados.
- **Número típico de hazards:** 7 a 10 em nível de sistema (não mais que uma dúzia).
- **Erros comuns:** confundir hazards com causas (falha de freio ≠ hazard), usar adjetivos ambíguos ("inseguro"), número excessivo.
- **Derivar restrições:** normalmente basta negar o hazard. Ex.: H = "aeronave viola separação mínima" → SC = "aeronave deve satisfazer separação mínima".
- **Template:** "Se [hazard] ocorrer, então [o que deve ser feito para prevenir/minimizar a perda]".
- **Rastreabilidade:** perdas ↔ hazards ↔ restrições (relacionamento muitos-para-muitos).

4.4 Passo 2 em Detalhe — Estrutura de Controle

- A estrutura é **funcional**, não física; é **hierárquica** (quem está acima tem mais autoridade).

- Não assume obediência — o fato de haver control action não garante que será executada.
- Um componente pode ser **simultaneamente** controlador e processo controlado (depende do ponto de vista).
- Inicie abstrata e refine iterativamente. O mínimo para avançar: 1 controlador + 1 control action + 1 processo controlado.
- Rótulos devem ser descritivos ("abrir válvula", "velocidade da roda"), não genéricos ("comando", "feedback").
- Atuadores e sensores são adicionados no Passo 4.

4.5 Passo 3 — UCAs (Ações de Controle Inseguras)

Tipo de UCA	Descrição
Não fornecida	A control action necessária não é fornecida quando deveria
Fornecida quando não devia	A control action é enviada em situação inadequada
Momento inadequado	A control action é fornecida muito cedo, muito tarde ou fora de ordem
Duração inadequada	A control action é aplicada por tempo insuficiente ou excessivo

4.6 Passo 4 — Cenários de Perdas (Falhas de Execução)

Falhas de execução ocorrem quando uma control action segura é enviada corretamente, mas não é implementada de forma eficaz no processo controlado. Causas: falha de atuador, falha de sensor, feedback incorreto, modelo de processo desatualizado, requisitos inadequados, erros de design.

5. Stakeholders e outros conceitos-chave

- **Stakeholder:** qualquer indivíduo, grupo ou organização com interesse legítimo nas decisões, resultados ou operação do sistema (direta ou indiretamente).
- **Usuário/Ator:** pessoa que utiliza o sistema. **Cliente (customer):** organização que adquire. **Acquirer:** entidade que efetua a aquisição.
- **Requirements Scrubbing:** selecionar o menor subconjunto viável de requisitos que atende os stakeholders.
- **Requisitos ambíguos:** aumentam o risco de entregar um sistema que não atende às necessidades reais dos usuários/stakeholders.
- **Prototipagem:** facilita o entendimento dos requisitos pelos stakeholders. Prototipagem de alta fidelidade pode inibir feedback.
- **Ferramentas de requisitos:** auxiliam na organização, rastreamento e controle de alterações ao longo do ciclo de vida.
- **Ações de controle na STPA:** comandos emitidos pelos controladores com objetivo de influenciar diretamente o comportamento do processo controlado.
- **Cenários com adversários (security):** incluídos na STPA para considerar falhas induzidas por agentes maliciosos (spoofing, alteração de comandos).
- **Capability Gap:** diferença entre a capacidade atual e a capacidade necessária para atingir os objetivos definidos.
- **Resultados da STPA:** subsídios para requisitos, orientações para arquitetura, recomendações de melhoria no projeto.

PARTE 2 — PROVA SIMULADA COMPLETA

Bacharelado em Ciência da Computação

Disciplina: Modelos e Técnicas Computacionais para Análise de Requisitos de Segurança (Safety e Security)

Valor: 10 pontos (peso 0.4)

Data: ____/____/____

Professor: Rodrigo Martins Pagliares

Instruções: Leia atentamente cada questão. Assinale apenas uma alternativa por questão. Preencha o gabarito abaixo antes de entregar. Boa prova!

Gabarito (preencha):

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
___	___	___	___	___	___	___	___	___	___	___	___	___	___	___	___	___	___	___	___

1. Assinale a alternativa CORRETA sobre a diferença entre requisitos de *safety* e de *security*:

- a) Requisitos de *safety* buscam garantir a confidencialidade de dados sensíveis, enquanto requisitos de *security* asseguram que o sistema permaneça em operação sem interrupções.
- b) Requisitos de *safety* são mais relevantes do que os de *security*, especialmente em sistemas que envolvem vidas humanas.
- c) Requisitos de *security* são formulados exclusivamente para ambientes financeiros e transacionais.
- d) Ambos os tipos de requisitos se referem às características de desempenho computacional, como tempo de resposta e latência.
- e) Requisitos de *safety* têm como foco a prevenção de perdas físicas, operacionais ou ambientais; requisitos de *security* lidam com proteção contra ameaças intencionais que comprometam integridade, disponibilidade ou confidencialidade do sistema.

2. Assinale a alternativa CORRETA sobre a diferença entre "perda" (*loss*) e "perigo" (*hazard*) na STPA:

- a) Perda é causada por falhas físicas de componentes, enquanto perigo representa falha de comunicação entre subsistemas de software.
- b) Perda corresponde diretamente a um acidente registrado; perigo é o mesmo que probabilidade de ocorrência.
- c) Perigo é definido como um tipo específico de falha elétrica que compromete o funcionamento do sistema; perda é a consequência ambiental do evento.
- d) Perda é um resultado indesejado para o sistema ou seus usuários; perigo é uma condição ou estado que pode levar à ocorrência dessa perda.
- e) Os termos são usados como sinônimos na STPA e se referem às consequências de eventos operacionais inesperados.

3. Assinale a alternativa CORRETA sobre a especificação de requisitos:

- a) Refere-se ao processo de documentação clara e consistente dos requisitos, com objetivo de facilitar a comunicação entre as partes envolvidas e permitir rastreabilidade.
- b) Deve ser estruturada de forma a evitar validações posteriores, priorizando o ganho de tempo no processo de desenvolvimento.
- c) Em métodos ágeis, a formalização dos requisitos é desnecessária, já que o processo se baseia em comunicação direta.
- d) Atua como substituta da fase de elicitação, uma vez que os requisitos já são conhecidos antes da análise do sistema.
- e) É uma atividade complementar que pode ser realizada após a implementação, com base nos testes realizados.

4. Assinale a alternativa CORRETA sobre modelos mentais no contexto da STPA:

- a) Têm como principal função representar as respostas esperadas dos sensores a estímulos do ambiente físico.
- b) Correspondem a comandos automáticos executados diretamente por dispositivos de controle baseados em hardware.
- c) São abstrações técnicas que substituem diretamente os algoritmos implementados em sistemas embarcados.
- d) Refletem a forma como controladores humanos percebem, interpretam e antecipam o comportamento do sistema, influenciando suas decisões.
- e) São considerados subjetivos e, por isso, não devem ser incluídos formalmente nas análises de sistemas complexos.

5. Assinale a alternativa CORRETA sobre requisitos funcionais:

- a) Referem-se aos aspectos visuais e de apresentação do sistema, como layout, cores e estilo da interface.
- b) Estabelecem regras de operação com foco em atributos de desempenho e confiabilidade sob diferentes condições de carga.
- c) Descrevem as funcionalidades específicas e os comportamentos que o sistema deve executar para atender às necessidades dos usuários.
- d) Incluem definições de componentes físicos e limitações de infraestrutura tecnológica que afetam a arquitetura do sistema.
- e) Relacionam-se a obrigações contratuais ou normativas, como leis e regulamentações que impactam o desenvolvimento do software.

6. Assinale a alternativa CORRETA sobre os tipos de ações de controle inseguras (UCAs) identificadas na STPA:

- a) São geralmente associadas à falha de sensores que enviam sinais incorretos ao sistema de controle.
- b) Compreendem ações de controle que não são fornecidas quando deveriam, são fornecidas quando não deveriam, ocorrem em momentos inadequados ou têm duração inadequada.
- c) Estão ligadas a situações em que a sequência esperada de comandos é violada por um erro de execução.
- d) Dizem respeito a instruções que exigem a ativação de componentes físicos com potencial de gerar impacto mecânico direto.
- e) Estão normalmente relacionadas a quedas de energia que impedem o envio contínuo de comandos.

7. Assinale a alternativa CORRETA sobre stakeholders:

- a) Incluem exclusivamente os usuários finais, pois são os únicos afetados diretamente pelo funcionamento do sistema.
- b) Englobam atores externos ao projeto que atuam em mercados concorrentes ou setores regulatórios.
- c) Dizem respeito prioritariamente aos financiadores do projeto, que exercem controle sobre prazos e orçamento.
- d) Compreendem os membros da equipe técnica, responsáveis por projetar, implementar e testar o sistema.
- e) Abrangem qualquer indivíduo, grupo ou organização com interesse legítimo nas decisões, resultados ou operação do sistema, seja de forma direta ou indireta.

8. Assinale a alternativa CORRETA sobre controladores humanos na STPA:

- a) São considerados elementos externos ao sistema analisado e, por isso, não são incluídos na modelagem da estrutura de controle.
- b) Costumam ser ignorados devido à dificuldade de prever o comportamento humano em sistemas complexos.
- c) São tratados como controladores ativos, com responsabilidades, modelos mentais e interações que influenciam diretamente o comportamento do sistema.
- d) São documentados apenas por meio de diagramas descritivos, sem representação funcional na estrutura de controle.
- e) Precisam ser substituídos por mecanismos automáticos para garantir consistência na análise de riscos.

9. Assinale a alternativa CORRETA sobre requisitos de security:

- a) Estão voltados à integridade física dos operadores e à prevenção de acidentes decorrentes de interação humana com o sistema.
- b) Focam na resistência mecânica de componentes e na minimização de falhas técnicas por desgaste ou sobrecarga.
- c) São definidos principalmente para aplicações sensíveis, como sistemas hospitalares e financeiros, onde a segurança da informação é considerada prioritária.
- d) Estabelecem mecanismos para proteger o sistema contra acessos não autorizados, alterações indevidas e falhas de disponibilidade, cobrindo aspectos de confidencialidade, integridade e disponibilidade.
- e) Estão relacionados a preferências estéticas, responsividade da interface e percepção subjetiva de facilidade de uso por parte do usuário final.

10. Assinale a alternativa CORRETA sobre os modelos de processo inadequados (*flawed process models*) na STPA:

- a) Estão associados a erros estatísticos em cálculos de otimização utilizados em sistemas adaptativos.
- b) Representam situações em que o controlador age com base em informações incompletas ou incorretas sobre o estado do sistema.
- c) Referem-se a defeitos físicos em subsistemas de energia que causam interrupções operacionais.
- d) Ocorrências desse tipo são características exclusivas de sistemas legados com baixa atualização tecnológica.
- e) Afetam apenas a documentação técnica do sistema, sem alterar seu comportamento real.

11. Assinale a alternativa CORRETA sobre prototipagem (*prototyping*) no contexto de requisitos:

- a) Facilita o entendimento dos requisitos por parte dos stakeholders, servindo como apoio visual e interativo ao processo de elicitação.
- b) Substitui completamente a necessidade de registrar os requisitos de forma estruturada e verificável.
- c) Funciona como alternativa à documentação tradicional e elimina a necessidade de validação formal.
- d) Aplica-se apenas a projetos que envolvem interfaces gráficas detalhadas e interação direta com o usuário final.
- e) Deve ser evitada em interações com stakeholders para manter a objetividade da análise.

12. Assinale a alternativa CORRETA sobre ações de controle na STPA:

- a) São comandos emitidos pelos controladores com o objetivo de influenciar diretamente o comportamento do processo controlado.
- b) Qualquer ação de controle enviada pelo sistema gera automaticamente uma situação de risco.
- c) Representam instruções executadas pelos atuadores em resposta a eventos externos.
- d) Correspondem a sinais de entrada gerados espontaneamente pelos sensores em sistemas reativos.
- e) São consideradas comunicações passivas sem impacto direto na operação do sistema.

13. Assinale a alternativa CORRETA sobre ferramentas de requisitos:

- a) Devem ser integradas apenas no final do desenvolvimento, quando os requisitos já estiverem estabilizados.
- b) Auxiliam na organização, rastreamento e controle das alterações dos requisitos ao longo do ciclo de vida do projeto.
- c) Tornam desnecessárias as interações presenciais com a equipe durante a coleta e validação de requisitos.
- d) São projetadas para uso exclusivo por profissionais da área de testes de software.
- e) Têm pouca utilidade em projetos de pequeno porte, devido à sua complexidade operacional.

14. Assinale a alternativa CORRETA sobre falhas de execução de ações de controle (*execution failures*) na STPA:

- a) São consideradas irrelevantes no contexto da STPA, pois essa técnica prioriza apenas falhas de concepção.
- b) Estão limitadas a erros cometidos por operadores humanos durante a interação com o sistema.
- c) Ocorrem quando uma ação de controle é enviada corretamente, mas não é implementada de forma eficaz no processo controlado.
- d) São neutras em termos de impacto, desde que o sistema esteja operando com energia e redundância ativa.
- e) Resultam exclusivamente de falhas de lógica em algoritmos de controle e não envolvem aspectos físicos.

15. Assinale a alternativa CORRETA sobre o conceito de *Capability Gap*:

- a) Refere-se à trajetória histórica de um requisito desde sua criação até a remoção.
- b) Designa o conjunto de funcionalidades mantidas por sistemas legados sem atualização.
- c) Está associado ao planejamento e execução de testes de regressão em ciclos de desenvolvimento contínuo.
- d) Trata-se de um tipo especial de requisito de usabilidade vinculado à experiência do usuário.
- e) É a diferença identificada entre a capacidade atual de um sistema e a capacidade necessária para atender aos objetivos definidos.

16. Assinale a alternativa CORRETA sobre como os resultados da STPA podem ser utilizados:

- a) São úteis apenas para fins investigativos após acidentes, mas não influenciam decisões de projeto preventivas.
- b) Direcionam a criação de testes de software de baixo nível, focados em coberturas estruturais do código.
- c) Servem como documentação complementar exigida por órgãos de auditoria, sem impacto direto na engenharia de sistemas.
- d) Fornecem subsídios para definição de requisitos, orientações para arquitetura e recomendações de melhoria no projeto.
- e) Substituem a modelagem comportamental baseada em notações como diagramas UML.

17. Assinale a alternativa CORRETA sobre o conceito de *requirements scrubbing*:

- a) Envolve a remoção completa de todos os requisitos que ainda não foram formalmente aprovados.
- b) Foca na identificação de stakeholders que possam estar em desacordo com os objetivos do sistema.
- c) Consiste em eliminar qualquer tipo de requisito que apareça mais de uma vez no documento técnico.
- d) Serve como alternativa à fase de testes de integração, reduzindo a necessidade de validação cruzada.
- e) Tem como finalidade selecionar e refinar o conjunto de requisitos mais consistente e viável para implementação.

18. Assinale a alternativa CORRETA sobre a inclusão de cenários com adversários (*adversarial scenarios*) na STPA:

- a) Não são considerados na STPA, pois essa técnica se aplica apenas à análise de acidentes não intencionais.
- b) São incluídos para considerar falhas induzidas por agentes maliciosos, como falsificação de sinais (*spoofing*) ou alteração de comandos.
- c) Devem ser avaliados exclusivamente após a fase de implementação e testes de segurança.
- d) São tratados como riscos externos que não precisam ser modelados dentro da estrutura funcional do sistema.
- e) São mapeados separadamente do processo de análise de perda, já que envolvem ameaças intangíveis.

19. Assinale a alternativa CORRETA sobre requisitos ambíguos:

- a) Podem ser desejáveis em projetos experimentais, pois oferecem espaço para interpretações múltiplas.
- b) Favorecem a produtividade da equipe ao permitir liberdade de implementação.
- c) Aumentam o risco de entregar um sistema que não atende às necessidades reais dos usuários ou stakeholders.
- d) Contribuem para facilitar ajustes futuros no escopo sem necessidade de revalidação.
- e) Auxiliam na identificação de otimizações durante a fase de codificação.

20. Assinale a alternativa CORRETA sobre a finalidade geral da STPA (*System-Theoretic Process Analysis*):

- a) Substituir completamente métodos baseados em confiabilidade estatística na engenharia de sistemas críticos.
- b) Localizar falhas raras de componentes que geram eventos isolados com impacto operacional limitado.
- c) Avaliar apenas falhas de dispositivos eletrônicos, negligenciando fatores humanos e organizacionais.
- d) Validar a conformidade da arquitetura do sistema com normas específicas, como a ISO 26262.
- e) Identificar condições inseguras que podem levar a perdas, mesmo quando todos os componentes do sistema funcionam corretamente.

GABARITO COMENTADO

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
E	D	A	D	C	B	E	C	D	B	A	A	B	C	E	D	E	B	C	E

Questão 1 — Resposta: E

Safety = ausência de perdas causadas por eventos não intencionais. Security = proteção da CIA contra ameaças intencionais. As alternativas a, b, c e d invertem, limitam ou distorcem as definições.

Questão 2 — Resposta: D

Hazard é um estado/condição do sistema que, em combinação com o pior caso do ambiente, leva a uma perda. Perda é o resultado indesejado. São conceitos distintos.

Questão 3 — Resposta: A

Especificação é a documentação dos requisitos com clareza e consistência, facilitando comunicação e rastreabilidade. Não substitui elicitação nem dispensa validações.

Questão 4 — Resposta: D

Modelo mental é o equivalente humano do process model: representa como o operador entende o estado atual do sistema e guia suas decisões de controle.

Questão 5 — Resposta: C

Requisitos funcionais descrevem o que o sistema deve fazer — funcionalidades e comportamentos. Aspectos de qualidade (b) e tecnologia (d) são não funcionais.

Questão 6 — Resposta: B

Os 4 tipos de UCAs são: (1) não fornecida; (2) fornecida quando não devia; (3) momento/ordem inadequados; (4) duração inadequada. A alternativa b é a definição canônica.

Questão 7 — Resposta: E

Stakeholders incluem clientes, usuários, desenvolvedores, órgãos reguladores, e qualquer parte interessada — definição ampla e inclusiva.

Questão 8 — Resposta: C

Na STPA, humanos são controladores de pleno direito, com modelos mentais (equivalente ao process model), algoritmos de decisão (procedimentos operacionais) e responsabilidades formais.

Questão 9 — Resposta: D

Security = CIA (Confidencialidade, Integridade, Disponibilidade). A alternativa d é a definição completa e correta. A alternativa c está parcialmente correta mas limitada a aplicações específicas.

Questão 10 — Resposta: B

Modelo de processo inadequado = controlador tem visão incorreta/desatualizada do estado real do sistema → emite control actions inseguras.

Questão 11 — Resposta: A

Prototipagem é uma técnica legítima de elicitação e validação. Protótipos de alta fidelidade podem inibir feedback; os de baixa fidelidade estimulam mais participação.

Questão 12 — Resposta: A

Control actions são comandos que os controladores emitem para influenciar o processo controlado. Não geram risco automaticamente — apenas quando são UCAs.

Questão 13 — Resposta: B

Ferramentas de requisitos (ex.: IBM DOORS, Jira) suportam organização, rastreabilidade e controle de alterações ao longo de todo o ciclo de vida.

Questão 14 — Resposta: C

Falhas de execução: a UCA foi correta, mas o processo controlado não a implementou adequadamente (ex.: falha de atuador, sensor com atraso).

Questão 15 — Resposta: E

Capability gap = lacuna entre o que o sistema consegue fazer hoje e o que é necessário para atingir os objetivos estratégicos. Ponto de partida do Design Conceitual.

Questão 16 — Resposta: D

Os resultados da STPA (UCAs, cenários, restrições) alimentam diretamente os requisitos de safety/security, decisões arquiteturais e planos de teste.

Questão 17 — Resposta: E

Requirements scrubbing = examinar cuidadosamente a especificação para selecionar o menor subconjunto de requisitos que atende as necessidades, eliminando complexidade desnecessária.

Questão 18 — Resposta: B

A STPA pode ser estendida para security. Cenários com adversários consideram ações maliciosas (spoofing, alteração de comandos) como causas de UCAs.

Questão 19 — Resposta: C

Ambiguidade é um dos maiores problemas em requisitos — leva a interpretações conflitantes e implementações incorretas, especialmente críticas em sistemas safety/security.

Questão 20 — Resposta: E

A STPA, baseada no STAMP, vai além das abordagens de confiabilidade (que assumem que acidentes = falhas de componentes). Ela identifica condições inseguras emergentes mesmo sem falhas individuais.